



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 8

Dr. Asad Mahmood

Feb. 03, 2026

Lecture Outline

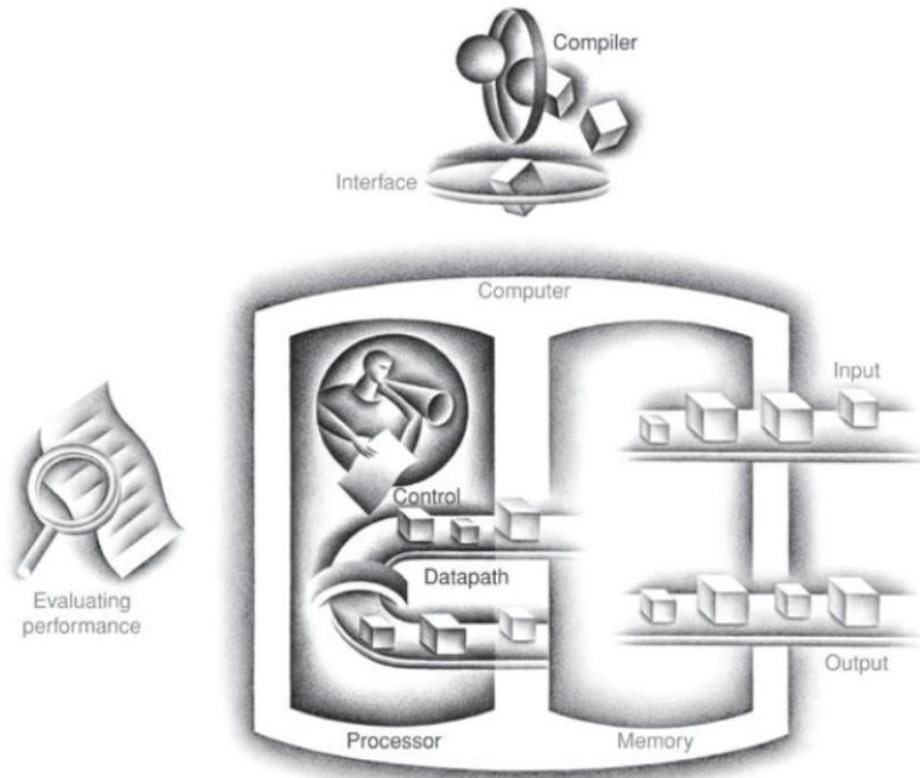
- Announcements (Assignment and Quiz 1 Day/Time/Venue)
- Outline of Previous Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
- Outline to Today's Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations

Chapter 2

Instructions: Language of the Computer

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - **2.3 Operands of the Computer Hardware**
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

RISC-V Operands

- Three types of operand are commonly used in the RISC-V Instruction Set / Assembly Language:
 1. Register Operands
 2. Memory Operands
 3. Immediate Operands

Register Operands

- Arithmetic instructions use **register** operands
- Registers are **small-sized, fastly-accessed memory locations** very close to CPU
- **RISC-V has a 32×64 -bit register-file**
 - Use for frequently accessed data
 - **32-bit data is called a “word”**
 - 64-bit data is called a **“doubleword”**
 - 32 x 64-bit general purpose registers x0 to x31
- *Design Principle 2: Smaller is faster*
 - Larger register-file size may increase delay

RISC-V Registers

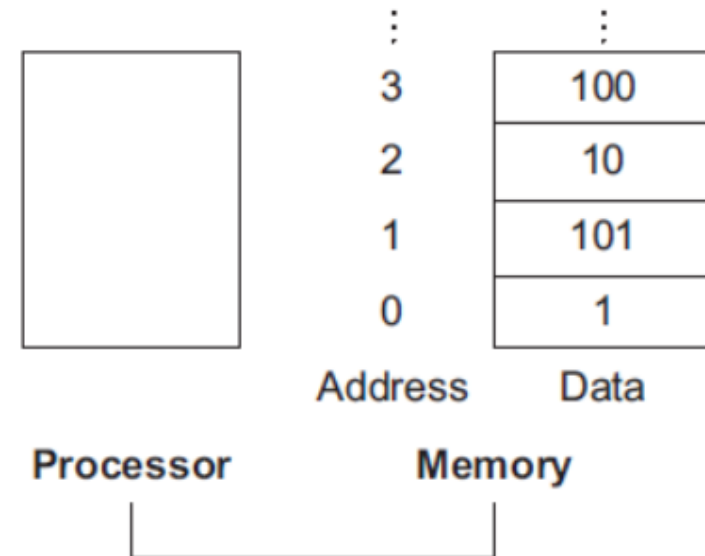
- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

Register Operand Example

- C code:
 $f = (g + h) - (i + j);$
 - $f, \dots, j$ in $x19, x20, \dots, x23$
- Compiled RISC-V code?

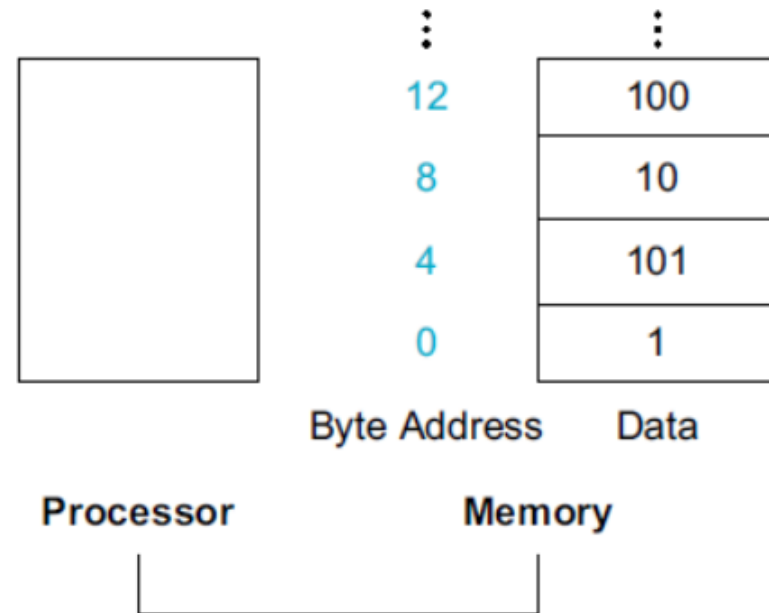
Memory Operands

- Main memory used to store more complex data-structures, e.g. Arrays, structures, dynamic data etc.
- To apply arithmetic operations
 - Data transfer instructions (load, store)
 - Load values from memory into registers
 - Store result from register to memory
- Memory is just a large, single-dimensional array, with the address acting as the index to that array, starting at 0.



Memory Operands

- Memory is byte addressed
 - Each address identifies an 8-bit byte
- RISC-V is Little Endian
 - Least-significant byte at least address of a word
 - *c.f.* Big Endian: most-significant byte at least address
- RISC-V does not require words to be aligned in memory
 - Unlike some other ISAs



Memory Operand Example

- Assume an array A of 100 words
- C code:
 $g = h + A[8];$
 - Variables g, h in registers x20 and x21 respectively
 - base address of A in register x22
- Compiled RISC-V code?
- Variant where the addition is stored in a memory location.

Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data require more instructions to be executed
- Compiler must use registers for variables as much as possible
 - Only spill to memory for less frequently used variables
 - Register optimization is important!
- Assuming 32-bit data, registers are roughly 200 times faster (0.25 vs. 50 nanoseconds) and are 10,000 times more energy efficient (0.1 vs. 1000 picoJoules) than DRAM in 2020.

Immediate Operands

- Constant data specified in an instruction

`addi x22, x22, 4`

- Make the common case fast
 - Small constants are common
 - Immediate operand avoids a load instruction
 - x0 dedicated for storing 0

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - **2.4 Signed and Unsigned Numbers**
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Binary Integers

- Computer work with numbers in their **binary format** -> **why?**
- Given an **n-bit number**
 - The corresponding decimal value?
 - Range?
 - Unsigned vs Signed
- RISC-V has both 32-bit and 64-bit variants
 - 32-bit/64-bit has correspondingly 32-bit/64-bit register width -> We will focus on 32-bit RISC-V variant

Signed Integers

- Different formats for representing signed nos.
 - Corresponding pros. and cons.?
- Given an n-bit 2's complement number
 - -ive nos. will always have a 1 in MSB -> can be considered as a 'sign' bit
 - The corresponding decimal value?
 - Range?
 - Overflow

Sign Extension

- Representing a number using more bits
 - Preserve the numeric value
- Replicate the sign bit to the left
 - c.f. unsigned values: extend with 0s
- Examples: 4-bit to 8-bit
 - $+2 \Rightarrow ?$
 - $-2 \Rightarrow ?$
- In RISC-V instruction set
 - `lb`: sign-extend loaded byte
 - `lbu`: zero-extend loaded byte

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - **2.5 Representing Instructions in the Computer**
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Representing Instructions

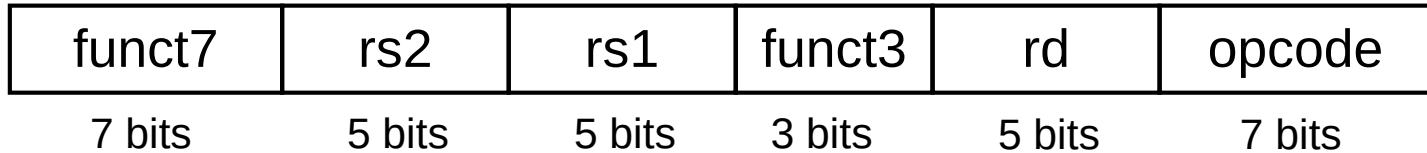
- There is a difference between the way humans instruct computers and the way computers see instructions.

add x9, x20, x21

0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Instructions are encoded in binary
 - Called machine code
- RISC-V instructions
 - Encoded as 32-bit instruction words
 - Field* and *instruction format*
 - Small number of formats
 - Simplicity favors Regularity!**

RISC-V R-format Instructions



■ Instruction fields

- opcode: Main operation (+format) of the instruction
- rd: destination register number
- funct3: 3-bit function code (additional opcode)
- rs1: the first source register number
- rs2: the second source register number
- funct7: 7-bit function code (additional opcode)

R-format Example

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

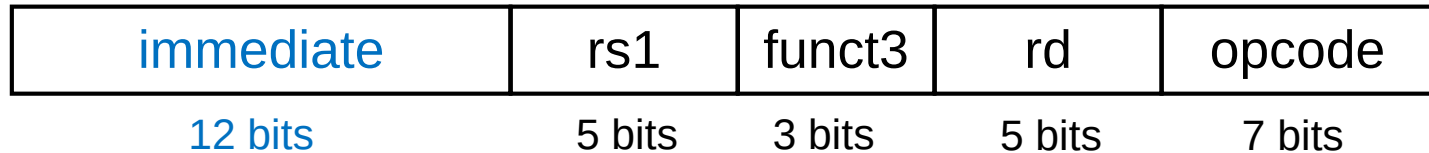
add x9, x20, x21

0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011

0000 0001 0101 1010 0000 0100 1011 0011_{two} =

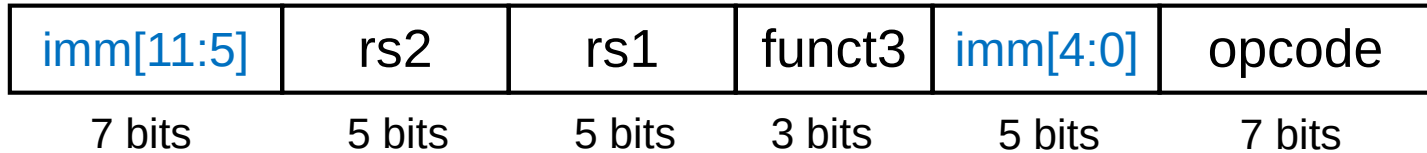
015A04B3₁₆

RISC-V I-format Instructions



- Immediate arithmetic and load instructions
 - rs1: source or base address register number
 - immediate: constant operand, or offset added to base address
 - 2s-complement, sign extended
- *Design Principle 3: Good design demands good compromises*
 - Different formats complicate decoding, but allow 32-bit instructions uniformly
 - Keep formats as similar as possible

RISC-V S-format Instructions



- Different immediate format for store instructions
 - rs1: base address register number
 - rs2: source operand register number
 - immediate: offset added to base address
 - Why Split?

RISC-V Opcodes for different Instruction Formats

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011
Instruction	Format	immediate		rs1	funct3	rd	opcode
addi (add immediate)	I	constant		reg	000	reg	0010011
lw (load word)	I	address		reg	010	reg	0000011
Instruction	Format	immed-iate	rs2	rs1	funct3	immed-iate	opcode
sw (store word)	S	address	reg	reg	010	address	0100011

RISC-V Instruction Formats

■ Example

immediate	rs1	funct3	rd	opcode
120	10	2	9	3
000011110000	01010	010	01001	0000011

funct7	rs2	rs1	funct3	rd	opcode
0	9	21	0	9	51
0000000	01001	10101	000	01001	0110011

immediate	rs1	funct3	rd	opcode
1	9	0	9	19
000000000001	01001	000	01001	0010011

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
3	9	10	2	24	35
0000011	01001	01010	010	11000	0100011